Overlay + Persist (v2)

The OS stays read-only. Only config.json, bridge.conf, the hostname and the eth0 IP are permanent - without changing existing code. Designed for this flow: one master is cloned to 40 cabinets; technicians only install and set the DIP switch.

WARNING - test on the master first

- Not yet tested on a physical Raspberry Pi. Only the scripts' logic was simulated on a computer (YAML sync, file rename, half-written YAML, stale file removal).
- One assumption must be proven on the master: NetworkManager writes `nmcli` IP changes to `/etc/netplan/90-NM-*.yaml`. Test 5 in section 5 proves it. If Test 5 fails, do not clone; send the result.
- Do not clone the master SD until `verify_overlay_persist.sh` shows no FAIL and all tests in section 5 pass.
- If unsure, use the standard Hardening package (A1).

1. What it does

With the overlay on, every write goes to RAM and the SD card is never written, so it cannot be damaged by power cuts. A few files are exceptions: whenever they change, a small root helper copies them to the real SD card (the lower layer at /media/root-ro) by re-opening it for writing for a few milliseconds and locking it again. Goal: no need to replace the SD card every month.

File / data	Behaviour with this package
config.json	Permanent (copied to the SD card on every change)
conf.d/bridge.conf + /etc/hostname	Permanent (hostname is correct at boot)
eth0 IP (netplan profile /etc/netplan/90-NM-*.yaml that defines eth0) **[NEW in v2]**	Permanent. Whenever dip.py changes the IP, the profile is copied to the SD card. The next boot uses the DIP-switch IP directly, never the image IP.
status.json, rfid1.json, rfid2.json	RAM only, revert to the image version at reboot (accepted). The session ID is also reset.
logs/	RAM only, lost at reboot
Wifi profile (wlan0) and all other /etc edits	Not permanent: revert to the image version at reboot

2. Why the IP must be permanent too

Without v2, the network profile changed by dip.py exists only in RAM. At every boot a cabinet starts with the image IP and dip.py changes it a few seconds later. As a result all 40 cabinets briefly share the same IP at every boot (especially after a power cut), and ipservice.py may read the image IP and rewrite bridge.conf (risk of a reboot loop).

Boot	Without v2 (IP only in RAM)	With v2 (IP permanent)
First boot of a cloned cabinet (DIP switch differs from the master)	Starts with the master IP, dip.py changes it.	Same (unavoidable): starts with the master IP. dip.py changes it, then the new IP is saved permanently. ipservice (after a delay) changes hostname + bridge.conf, saves them, reboots once.
Later boots, including after a power cut	Starts with the master IP again at every boot; risk of IP collisions and reboot loops.	Boots directly with the DIP-switch IP. dip.py changes nothing, ipservice.py finds no change, no reboot.

3. Package contents

File	Purpose
overlay_persist/sitecustomize.py	Unchanged. Atomic writes; after config.json / bridge.conf is written under overlay, calls the helper.
overlay_persist/broker-persist	**Updated (v2).** Root helper. Accepts only: config.json, conf.d/bridge.conf, network. Refuses symlinks, validates JSON and YAML, writes atomically to the real SD card. Copies the eth0 YAML only; the wifi YAML is never touched.
setup_overlay_persist.sh	**Updated.** Installs the helper, restricted sudo, systemd drop-ins, the ipservice start delay (30 s, adjustable) and the IP-profile saver (broker-persist-net .path + .timer). Safe to re-run. Does not enable the overlay.
verify_overlay_persist.sh	**NEW.** Read-only. Checks the installation and compares the real SD card with the current state (config, bridge.conf, hostname, eth0 IP). Run on the master before cloning.
update_broker.sh	Unchanged. Updates broker code while the overlay is on.
remove_overlay_persist.sh	**Updated.** Removes everything, including the IP-profile saver.

4. Preparing the master (done by you, not the technician)

1. Disable the overlay if it is on, then check: `findmnt -no FSTYPE /` must print **ext4**.
2. **Set the master's DIP switch to the master's IP** (e.g. master .101 means the switch for .101). Otherwise the master changes its own IP at every boot.
3. Copy the contents of B_OVERLAY_PERSIST to /home/alice/broker/ (overwrite the old package B files: broker-persist, setup_overlay_persist.sh, remove_overlay_persist.sh), then install:

```
cd /home/alice/broker
chmod +x *.sh overlay_persist/broker-persist
sudo ./setup_overlay_persist.sh
sudo systemctl restart send ipservice
sudo ./verify_overlay_persist.sh      # overlay off: there must be no FAIL
```

4. Enable the overlay:

```
sudo raspi-config nonint do_overlayfs 0
sudo reboot
```

5. After the boot, check (if /media/root-ro is missing, the package cannot be used: disable the overlay, section 8):

```
findmnt -no FSTYPE /          # overlay
findmnt /media/root-ro         # real SD card (read-only), must exist
sudo ./verify_overlay_persist.sh   # all OK, no FAIL
```

5. Tests on the master (all must pass)

#	Test	Expected result
1	Change a setting (config.json) from the web page, wait for the automatic reboot.	The new setting is still present after the reboot.
2	Change the setting again, cut power 1-2 seconds later; repeat 10-20 times.	config.json is always valid JSON (python3 -m json.tool), holding the old or the new value.
3	Send a scan, reboot.	rfid1/rfid2 revert to the image version (accepted); the broker keeps working.
4	Measure SD writes during normal operation (section 9).	Almost no sectors written while scanning and with the logger on.
5	**Change the DIP switch from the master IP (.101) to another IP (e.g. .102)** with the overlay on. Wait for the automatic reboot (at most one). Then: `ip -4 a show eth0; hostname; ls -l /etc/netplan; sudo ./verify_overlay_persist.sh`	IP is .102, hostname and bridge.conf match .102, the 90-NM-*.yaml timestamp changed, verify shows no FAIL. The Pi reboots at most once. **If /etc/netplan did not change: stop and send the result.**
6	From a PC, ping the old IP (.101) continuously, then reboot the Pi manually (`sudo reboot`).	No reply from .101 during boot; the Pi answers on .102 straight away. No extra reboot.
7	After Test 5, cut power and power on again (repeat 3 times).	Boots straight to .102 every time; hostname and bridge.conf stay; no repeated reboots.
8	Change the switch to a third IP (e.g. .105), wait, then back to .102.	Every change is saved; no duplicate eth0 YAML files in /etc/netplan; verify shows no FAIL.
9	**Real clone test:** after the master is restored (section 6) and imaged, write the image to one other SD, put it in another Pi (or the same Pi), set the switch to a different IP, power on.	Exactly what the technician will do: one automatic reboot, then IP/hostname/bridge.conf correct; a power cut afterwards is still correct.
10	Check the helper log: `journalctl -t broker-persist`	'persisted ...' lines only when config.json, bridge.conf or the IP profile changed.

Quick check that the IP saver works: `sudo diff /etc/netplan/90-NM-*.yaml /media/root-ro/etc/netplan/90-NM-*.yaml` (compare the eth0 file with its counterpart; there must be no difference, at most one minute after the IP changed).

Remote Test 5 (without touching the switch): run `sudo ./test_ip_persist.sh`. It mimics dip.py's commands with test IP .199, checks that the IP lands in /etc/netplan and is copied to the real SD, then restores the original IP. It runs in the background (the session may drop for ~30 s); read the result with `cat /tmp/iptest.log` after about 4 minutes. It can run before or after the overlay is enabled. Boot test: `sudo ./test_ip_persist.sh --reboot`, then `sudo journalctl -u dip.service -b | grep 'Current IP'` must show the test IP. This does not replace the real clone test with the switch (Test 9), but it proves the core assumption.

6. Restore the master to its initial state, then make the image

Anything saved permanently during testing is cloned too, so the master must be restored before imaging:

1. Set the DIP switch back to the master IP (.101). Wait for the automatic reboot.
2. Check hostname = the master's name (e.g. MASCAB01A) and bridge.conf = the master's name. **Note:** if the master's hostname currently shows MASCAB01B or another letter, the master is not in its initial state.
3. `sudo ./verify_overlay_persist.sh` must show no FAIL, and `findmnt -no FSTYPE /` must print overlay.
4. Shut down properly: `sudo poweroff`. Wait until the LED stops blinking, then remove the SD.
5. Make the image from that SD on a computer (never clone from a running Pi). Use target SDs of the same size or larger.

7. The 40-cabinet flow

Stage	Who	What happens
Preparation	You / SD preparer	Write the master image to 40 SDs (overlay and package B already included). Install the SDs in the cabinets.
Installation	Technician	Set the DIP switch to the cabinet number, power on, wait for one automatic reboot. See B2 (technician sheet).
Afterwards	Automatic	IP, hostname and bridge.conf are permanent. Power cuts or later switch changes are handled automatically (switch change: one reboot if the cabinet name changes).

Brief IP collision (once per cabinet): on the first boot, a cloned cabinet briefly uses the master IP (.101). If the real .101 cabinet is running, its connection may be disturbed for a few seconds. Therefore technicians are asked to power cabinets on one at a time and wait for the automatic reboot to finish before the next one. Another way to remove the collision: make the master IP a spare IP that no cabinet uses (adjust the master's switch accordingly).

8. Updating the broker, disabling the overlay

Update with the overlay on (config.json, status.json, rfid*.json, bridge.conf, logs/ and cloud/ are not overwritten; the new code is active after a reboot):

```
# 1. extract the new broker.zip to /home/alice/new_broker (the folder that contains apps.py)
sudo ./update_broker.sh /home/alice/new_broker
sudo reboot
```

New keys added to config.json in a newer version are not applied automatically, because config.json is never overwritten. Disable the overlay (and uninstall):

```
sudo raspi-config nonint do_overlayfs 1
sudo reboot
# after boot (ext4):
sudo ./remove_overlay_persist.sh      # optional uninstall
```

Changing the DIP switch with the overlay on is now safe and automatic (v2); there is no need to disable the overlay.

9. Risks and limitations

Item	Risk	Notes
Assumption: nmcli writes IP changes to /etc/netplan/90-NM-*.yaml	Medium - untested	Your output shows the eth0 profile comes from netplan and 90-NM-*.yaml files exist. Test 5 proves that dip.py's change lands in that file. If not, v2 cannot be used.
IP collision at first boot of a clone	Medium, once per cabinet	See section 7. Power on one at a time.
Master DIP switch differs from the master IP	High if forgotten	The master changes its own IP at every boot and may be cloned in a wrong state. Check section 4 step 2 and section 6.
ipservice start delay (30 s)	Low	A backup so ipservice does not read a temporary IP. ipservice starts 30 s later at every boot. Change with: sudo IPSERVICE_DELAY=45 ./setup_overlay_persist.sh
Power cut during a clone's first boot	Low	The process repeats itself: no data is damaged, just one extra setup round.
Short remount window	Low-Medium	A power cut during the few milliseconds the SD card is writable can affect the OS partition. Only when config.json, bridge.conf or the IP changes; far rarer than continuous logging, but not zero.
Backup timer every minute	Very low	Only reads and compares files; the SD card is not written when nothing changed.
Assumption: real SD card at /media/root-ro	Low	Standard for raspi-config overlay; verify in section 4.
Wifi profile and other OS changes revert at reboot	Certain	Only the files in section 1 persist. If wifi is used in the field, the profile must already be in the image.
Unattended behaviour untested	Unknown	Not tested on a physical Raspberry Pi with the full application. Test on the master, then 2-3 cabinets, before all 40.

10. If the Pi reboots repeatedly, and measuring SD writes

- Log in over SSH between two reboots, run `sudo raspi-config nonint do_overlayfs 1`, then `sudo reboot`.
- If SSH is unreachable: put the SD in a PC, open cmdline.txt on the bootfs drive with a plain text editor and remove the overlay entry (e.g. overlayroot=tmpfs); the file must remain one line.
- Always keep an SD card image backup before enabling the overlay.

Measure SD writes (goal: no SD replacement every month). Read the sector counter, wait 10 minutes of normal operation, read again; the difference should be close to zero. Run the same on the current setup (without overlay) as a comparison:

```
awk '{print $7}' /sys/block/mmcblk0/stat   # sectors written (512 bytes each)
sleep 600
awk '{print $7}' /sys/block/mmcblk0/stat
```